

智能工程实验大作业

基于 ROS2 的移动机器人自主建图与多点导航系统

组 员：杨子源 刘振毫 许佳明

时 间：2026.6.3

摘 要

本项目基于 ROS2 和 Gazebo 仿真环境，以 TurtleBot3 机器人为平台，实现了未知环境下的自主探索建图与已知环境下的多点自主导航功能。系统集成了 slam_toolbox 在线异步 SLAM 算法、Nav2 导航栈以及基于前沿探索的 explore_lite 算法，并通过自定义 ROS2 节点实现了定点巡航逻辑。在自主建图阶段，系统利用激光雷达和里程计数据实时构建二维栅格地图，explore_lite 节点通过广度优先搜索提取前沿区域并驱动机器人完成全图探索；在多点导航阶段，系统采用 AMCL 算法进行全局定位，结合代价地图实现全局路径规划与局部避障，自定义控制节点按照预设目标点序列依次完成导航任务。实验结果表明，系统能够在仿真环境中稳定完成自主建图与多点巡航，验证了各模块集成的有效性与流程自动化的可行性。

关键词：ROS2；自主建图；前沿探索；多点导航；TurtleBot3

目录

1	实验目的	3
2	实验环境与工具	3
2.1	软件环境	3
2.2	硬件平台	3
2.3	仿真环境	4
3	系统总体设计	4
3.1	系统架构	4
3.2	工作空间结构	4
3.3	功能包说明	4
3.3.1	course_demo 功能包	4
3.3.2	m-explore-ros2 功能包	5
4	核心算法与实现	5
4.1	基于前沿探索的自主建图	5
4.1.1	SLAM 建图原理	5
4.1.2	前沿探索算法	5
4.1.3	代价地图客户端	7
4.2	基于 Nav2 的多点自动巡航	7
4.2.1	Nav2 导航栈	7
4.2.2	多点导航控制节点	7
4.2.3	简单避障探索节点	8
5	系统集成与流程控制	9
5.1	Launch 文件设计	9
5.2	Shell 脚本封装	9
5.3	Explore Lite 参数配置	9
6	实验过程与结果	10
6.1	实验步骤	10
6.2	自主建图结果	10
6.3	多点导航结果	10
7	实验分析与讨论	11

7.1	前沿探索算法分析	11
7.2	导航系统分析	11
7.3	系统集成问题与解决	11
8	实验总结	12

第一章 实验目的

本实验旨在基于 ROS2 机器人操作系统和 Gazebo 物理仿真环境，设计并实现一个完整的移动机器人自主建图与多点导航系统。通过本实验，掌握 ROS2 工作空间的创建、功能包的组织与 colcon 编译流程，理解并实践基于 slam_toolbox 的 2D SLAM 建图原理与在线异步建图模式，学习前沿探索算法的工作原理以实现未知环境下的自主探索建图，掌握 Nav2 导航栈的配置与使用，通过自定义 ROS2 节点实现多点序列导航的业务逻辑控制，并综合运用 Launch 文件、Shell 脚本等工具实现系统各模块的自动化启动与流程控制。

第二章 实验环境与工具

第一节 软件环境

本实验所采用的软件环境如表1所示。系统运行于 Ubuntu 24.04 操作系统，采用 ROS2 Jazzy Jalisco 发行版，仿真引擎为 Gazebo Harmonic。机器人平台选用 TurtleBot3 Burger 构型，SLAM 算法采用 slam_toolbox 的在线异步模式，导航框架为 Nav2，探索算法使用 explore_lite。编程语言涉及 Python 3.12 与 C++17，构建工具为 colcon。

表 1: 软件环境配置

组件	版本/说明
操作系统	Ubuntu 24.04
ROS2 发行版	ROS2 Jazzy Jalisco
仿真引擎	Gazebo Harmonic
机器人平台	TurtleBot3 Burger
SLAM 算法	slam_toolbox
导航框架	Nav2
探索算法	explore_lite
编程语言	Python 3.12 / C++17
构建工具	colcon

第二节 硬件平台

本实验采用 TurtleBot3 Burger 构型作为仿真机器人平台。该平台尺寸为 138mm × 178mm × 192mm，采用双轮差速驱动方式，搭载 360 度 2D 激光雷达传感器，扫描半径为 3.5m。计算平台为 Raspberry Pi，最大线速度为 0.22m/s，最大角速度为 2.84rad/s。

第三节 仿真环境

实验采用 Gazebo 提供的 TurtleBot3 World 标准仿真场景。该场景为一个封闭式矩形平面区域，外围由实体墙壁界定，内部非均匀散布六棱柱、圆柱和立方体等静态障碍物。障碍物的排列形成了回环通道、不规则转角及开阔区域，提供了丰富的几何特征，有利于验证 2D 激光雷达的空间扫描性能及 SLAM 算法的局部特征匹配与回环检测精度。

第三章 系统总体设计

第一节 系统架构

本系统采用模块化的分层架构设计，整体分为仿真层、感知定位层、建图探索层、导航规划层和应用控制层五个层次，如图1所示。最底层为仿真层，由 Gazebo 和 TurtleBot3 Burger 构成，提供物理仿真与传感器数据；感知定位层负责激光雷达数据、里程计数据与 TF 坐标变换的处理；建图探索层包含 slam_toolbox 和 explore_lite，分别负责实时建图与自主探索；导航规划层为 Nav2 导航栈，涵盖 AMCL 定位、全局规划器、局部控制器与代价地图；最上层为应用控制层，由 auto_navigator 和 auto_explorer 节点实现业务逻辑。

应用控制层：auto_navigator / auto_explorer
导航规划层：Nav2
建图探索层：slam_toolbox + explore_lite
感知定位层：LiDAR / Odometry / TF
仿真层：Gazebo + TurtleBot3 Burger

图 1: 系统分层架构示意图

第二节 工作空间结构

项目工作空间 ros2_project_ws 的核心目录结构如下：scripts/目录包含 4 个流程控制 Shell 脚本，分别用于启动建图与自主探索、保存栅格地图、启动导航与多点巡航以及独立启动探索算法；src/目录包含自定义功能包 course_demo 以及第三方前沿探索功能包 m-explore-ros2；maps/目录存放保存的栅格地图数据；build/和 install/分别为编译输出和安装输出目录。

第三节 功能包说明

3.3.1 course_demo 功能包

course_demo 是本项目的自定义 ROS2 功能包，承担系统集成与业务逻辑控制的核心职责。其 Launch 文件包含 slam_demo.launch.py 和 nav_demo.launch.py，前者组合启动 Gazebo 仿真、SLAM Toolbox、Nav2 导航栈及 Explore Lite 节点，后者组合启动

Gazebo 仿真、Nav2 导航栈及 RViz2 可视化界面。Python 节点方面，`auto_navigator.py` 实现了多点序列导航控制逻辑，`auto_explorer.py` 实现了基于激光雷达的简单避障探索功能。参数配置方面，`burger.yaml` 存储 Nav2 参数，`explore.yaml` 存储 Explore Lite 参数。

3.3.2 m-explore-ros2 功能包

`m-explore-ros2` 是引入的开源探索功能包，其核心节点 `explore_lite` 实现了基于前沿的自主探索算法。该功能包采用 C++ 编写，主要包含三个模块：`Explore` 节点负责前沿搜索调度、导航目标发送与探索状态管理；`FrontierSearch` 模块实现基于广度优先搜索的前沿检测与代价计算；`Costmap2DClient` 模块负责代价地图数据的订阅与更新管理。

第四章 核心算法与实现

第一节 基于前沿探索的自主建图

4.1.1 SLAM 建图原理

系统采用 `slam_toolbox` 的在线异步建图模式，利用激光雷达扫描数据与轮式里程计数据，通过基于图优化的 SLAM 方法实时构建二维占用栅格地图。其核心流程为：首先根据轮式编码器数据推算机器人位姿先验，然后将当前激光扫描与已有地图进行匹配以优化位姿估计，接着构建位姿图并通过回环检测约束进行全局优化，最后将优化后的位姿与扫描数据融合以更新栅格地图。

4.1.2 前沿探索算法

前沿是指已探索自由区域与未探索未知区域的交界线。前沿探索算法的核心思想是不断驱动机器人前往最近的前沿区域，从而逐步扩展已知地图，直至环境中不再存在可达的未知区域。

前沿检测算法基于广度优先搜索，从机器人当前位置出发遍历代价地图，识别满足以下条件的栅格单元作为前沿单元：该单元的状态为 `NO_INFORMATION`，且该单元的 4 邻域中至少存在一个 `FREE_SPACE` 单元。检测到前沿单元后，算法通过 8 邻域 BFS 将相邻的前沿单元聚合为前沿区域，并计算每个前沿区域的质心、最近距离和规模。核心代码如下：

```
1 std::vector<Frontier>
2 FrontierSearch::searchFrom
3 {
4     std::vector<Frontier> frontier_list;
5     std::queue<unsigned int> bfs;
6     // initialize BFS from nearest free cell
7     while ) {
```

```

8   unsigned int idx = bfs.front;
9   bfs.pop;
10  for ) {
11      if {
12          visited_flag[nbr] = true;
13          bfs.push;
14      } else if ) {
15          frontier_flag[nbr] = true;
16          Frontier new_frontier = buildNewFrontier;
17          if
18              >= min_frontier_size_) {
19              frontier_list.push_back;
20          }
21      }
22  }
23  }
24  for {
25      frontier.cost = frontierCost;
26  }
27  std::sort, frontier_list.end,
28  []
29  { return f1.cost < f2.cost; });
30  return frontier_list;
31  }
    
```

前沿检测核心代码

每个前沿区域的代价由以下公式计算：

$$C = \alpha \cdot d_{\min} \cdot r - \beta \cdot s \cdot r \quad (1)$$

其中, $d_{\min}$ 为前沿到机器人的最小距离, s 为前沿的规模, r 为地图分辨率, α 为距离权重, β 为增益权重。该代价函数在距离与信息增益之间取得平衡：距离越近代价越低, 规模越大代价越低。本实验中, 参数配置为 $\alpha = 1.0$, $\beta = 3.0$, 最小前沿尺寸为 0.75m, 规划频率为 2.0Hz。

Explore 节点的主控逻辑由定时器驱动, 按规划频率周期性执行 makePlan 函数。其流程为：首先获取机器人当前位姿, 然后调用 FrontierSearch::searchFrom 搜索所有前沿区域；若无前沿则发布探索完成状态并停止；否则选择代价最小的非黑名单前沿作为目标, 通过 Nav2 Action Client 发送 NavigateToPose 目标；同时监控导航进度, 若超时无进展则将目标加入黑名单；导航完成后立即重新规划。

```

1 void Explore::makePlan
2 {
3     auto pose = costmap_client_.getRobotPose;
4     auto frontiers = search_.searchFrom;
5     if ) {
6         stop;
7         return;
    
```



```

8   }
9   auto frontier = std::find_if_not,
10    frontiers.end,
11    [this] {
12        return goalOnBlacklist;
13    });
14   if ) {
15       stop;
16       return;
17   }
18   auto goal = nav2_msgs::action::NavigateToPose::Goal;
19   goal.pose.pose.position = frontier->centroid;
20   move_base_client_->async_send_goal;
21 }

```

探索主控逻辑

4.1.3 代价地图客户端

Costmap2DClient 类负责订阅和更新代价地图数据。它订阅/map 话题获取完整的占用栅格地图，并订阅/map_updates 话题获取增量更新。数据接收后，通过线性映射将 OccupancyGrid 的 [0, 100] 范围转换为代价地图的 [0, 255] 范围：

$$C_{\text{costmap}} = 1 + \frac{251 \times c_{\text{grid}}}{97} \quad (2)$$

其中特殊值为： $c_{\text{grid}} = 0$ 映射为 0， $c_{\text{grid}} = 99$ 映射为 253， $c_{\text{grid}} = 100$ 映射为 254， $c_{\text{grid}} = -1$ 映射为 255。

第二节 基于 Nav2 的多点自动巡航

4.2.1 Nav2 导航栈

Nav2 是 ROS2 的标准导航框架，本实验中使用的 Nav2 导航栈包含以下核心组件：AMCL 基于粒子滤波的概率定位算法，通过激光扫描与已知地图的匹配估计机器人位姿；全局代价地图基于静态地图构建全局代价信息，用于全局路径规划；局部代价地图基于传感器实时数据提供局部代价信息，用于局部避障；全局规划器基于代价地图计算从起点到目标的全局路径；局部控制器根据全局路径和局部代价地图输出速度指令，实现路径跟踪与避障；行为服务器处理导航过程中的恢复行为。

4.2.2 多点导航控制节点

自定义控制节点 auto_navigator.py 实现了多点序列导航的业务逻辑。启动后，节点首先通过 API 发送初始位姿 (0.0, 0.0, 0.0) 完成 AMCL 定位系统的初始化，然后调用 waitUntilNav2Active 阻塞等待导航栈完全启动。随后，程序按预设的四个目标点坐标依次发送导航请求并阻塞等待结果，同时发布 MarkerArray 消息在 RViz2 中显示目标

点位置与序号。四个预设目标点如表2所示。

```

1 # Set initial pose
2 initial_pose = PoseStamped
3 initial_pose.header.frame_id = 'map'
4 initial_pose.pose.position.x = 0.0
5 initial_pose.pose.position.y = 0.0
6 initial_pose.pose.orientation.w = 1.0
7 navigator.setInitialPose
8 navigator.waitUntilNav2Active
9
10 # Predefined waypoints
11 goals = [ , , , ]
12
13 # Sequential navigation
14 for idx, in enumerate:
15     goal = PoseStamped
16     goal.header.frame_id = 'map'
17     goal.pose.position.x = x
18     goal.pose.position.y = y
19     goal.pose.orientation.w = 1.0
20     navigator.goToPose
21     while not navigator.isTaskComplete:
22         feedback = navigator.getFeedback
23         time.sleep
24     result = navigator.getResult

```

多点导航核心代码

表 2: 预设巡航目标点

序号	坐标 (x, y) / m	说明
1	(1.5, 1.5)	左上方区域
2	(2.0, -1.5)	左下方区域
3	(4.0, 1.0)	右上方区域
4	(0.0, 0.0)	返回原点

4.2.3 简单避障探索节点

此外，项目还实现了 auto_explorer.py 节点，采用基于激光雷达的简单反应式避障策略。该节点订阅 /scan 话题获取激光扫描数据，当正前方 30 度范围内检测到障碍物距离小于 0.6m 时，切换为旋转避障状态；否则保持前进并附加随机角速度扰动以增加探索覆盖范围。

```

1 def scan_callback:
2     front_ranges = msg.ranges[-30:] + msg.ranges[:30]
3     valid_ranges = [r for r in front_ranges
4                     if r > 0.1 and r < float]
5     if len > 0:
6         min_dist = min

```

```

7         if min_dist < 0.6:
8             if self.state == 'FORWARD':
9                 self.state = 'TURN'
10                self.turn_time = random.randint
11            else:
12                if self.state == 'TURN' and self.turn_time <= 0:
13                    self.state = 'FORWARD'

```

简单避障探索代码

第五章 系统集成与流程控制

第一节 Launch 文件设计

系统通过两个 Launch 文件实现不同阶段的模块组合启动。`slam_demo.launch.py` 用于自主建图阶段，其依次启动 Gazebo 仿真环境、SLAM Toolbox 在线异步建图节点和 Nav2 导航栈，延迟 3 秒启动 RViz2 可视化界面，延迟 8 秒启动 Explore Lite 前沿探索节点。延迟启动策略确保了各模块按依赖关系依次初始化，避免因资源竞争导致的启动失败。`nav_demo.launch.py` 用于多点导航阶段，首先声明地图文件路径参数，然后启动 Gazebo 仿真环境和 Nav2 导航栈，延迟 5 秒启动 RViz2 可视化界面。

第二节 Shell 脚本封装

为简化操作流程，项目通过 4 个 Shell 脚本封装了完整的实验流程。`1_mapping.sh` 负责配置环境变量，`source` 工作空间，清理残留 Gazebo 进程，并启动 `slam_demo.launch.py`。`2_save_map.sh` 调用 `nav2_map_server` 的 `map_saver_cli` 工具，将 SLAM 生成的栅格地图保存至 `maps/` 目录。`3_navigation.sh` 首先检查地图文件是否存在，然后启动 `nav_demo.launch.py`，等待 15 秒后启动 `auto_navigator` 节点。`4_auto_explore.sh` 为独立启动探索算法节点的辅助脚本。

第三节 Explore Lite 参数配置

Explore Lite 节点参数配置如表3所示。

表 3: Explore Lite 参数配置

参数名	值	说明
robot_base_frame	base_link	机器人基座坐标系
return_to_init	true	探索完成后返回起点
costmap_topic	map	代价地图话题
visualize	false	是否可视化前沿
planner_frequency	2.0	规划频率
progress_timeout	5.0	进度超时
potential_scale	1.0	距离权重 α
gain_scale	3.0	增益权重 β
min_frontier_size	0.75	最小前沿尺寸
transform_tolerance	0.3	TF 变换容差

第六章 实验过程与结果

第一节 实验步骤

本实验按照以下步骤依次执行:首先在 `ros2_project_ws` 目录下执行 `colcon build` 编译所有功能包;然后执行 `bash scripts/1_mapping.sh` 启动自主建图,系统自动启动 Gazebo 仿真、SLAM Toolbox、Nav2 导航栈和 Explore Lite 节点,机器人开始自主探索未知环境并实时构建栅格地图;待探索完成后,执行 `bash scripts/2_save_map.sh` 将构建的地图保存为 `my_map.yaml` 和 `my_map.pgm`;最后执行 `bash scripts/3_navigation.sh` 启动多点导航,系统加载已保存的地图,启动 Nav2 导航栈,并自动运行 `auto_navigator` 节点,机器人依次前往四个预设目标点完成巡航任务。

第二节 自主建图结果

在自主建图阶段,Explore Lite 节点成功检测到环境中的前沿区域并驱动机器人逐一前往探索。机器人从原点出发,按照前沿代价从小到大的顺序依次访问各前沿区域,最终完成了对 TurtleBot3 World 仿真环境的完整建图。建图过程中,SLAM Toolbox 实时更新栅格地图,地图质量良好,墙壁轮廓清晰,障碍物位置准确。Explore Lite 节点的探索状态通过 `explore/status` 话题发布,当所有前沿区域均被探索后,节点发布 `EXPLORATION_COMPLETE` 状态并自动停止。由于配置了 `return_to_init: true`,机器人随后自动返回初始位姿。

第三节 多点导航结果

在多点导航阶段,`auto_navigator` 节点成功完成了四个预设目标点的序列导航。机器人从原点出发,沿规划路径顺利到达左上方区域目标点 1,导航状态为 `SUCCEEDED`;随后从目标点 1 出发绕过中间障碍物到达左下方区域目标点 2,导航状态为 `SUC-`

CEEDED; 接着从目标点 2 出发穿越开阔区域到达右上方区域目标点 3, 导航状态为 SUCCEDED; 最后从目标点 3 返回原点目标点 4, 导航状态为 SUCCEDED。导航过程中, RViz2 界面实时显示机器人位姿、全局路径、局部代价地图和目标点标记, AMCL 定位稳定, 路径规划合理, 局部避障有效。

第七章 实验分析与讨论

第一节 前沿探索算法分析

本实验采用的前沿探索算法具有良好的完备性, 在连通环境中能够保证对所有可达区域的探索, 因为只要存在未探索区域就一定存在前沿。代价函数综合考虑了距离和信息增益, 使机器人优先探索近距离的大面积未知区域, 提高了探索效率。黑名单机制和进度超时机制有效处理了导航失败和局部最优问题, 当某个前沿目标无法到达时, 算法会将其加入黑名单并尝试其他前沿, 增强了鲁棒性。然而, 前沿探索算法属于贪心策略, 不保证全局最优的探索路径, 在大型复杂环境中可能存在重复探索已访问区域的情况, 这是其局限性所在。

第二节 导航系统分析

Nav2 导航栈在本实验中表现稳定。AMCL 定位在特征丰富的环境中精度较高, 但在长走廊等几何特征匮乏的区域可能出现粒子退化, 需要足够的初始粒子数和合理的初始位姿估计。全局规划器基于 Dijkstra 或 A* 算法, 能够生成无碰撞的最短路径; 局部控制器基于 DWB 算法, 能够实时避障并跟踪路径。全局代价地图基于静态地图构建, 局部代价地图基于实时传感器数据更新, 两者结合实现了全局规划与局部避障的协调。当导航失败时, Nav2 会触发旋转清除等恢复行为, 但在某些情况下可能导致机器人偏离预期路径。

第三节 系统集成问题与解决

在系统集成过程中遇到了若干主要问题。首先是模块启动时序问题, 各功能模块存在依赖关系, 若同时启动可能导致资源竞争或连接失败, 解决方案是采用 TimerAction 实现延迟启动, 确保底层模块先于上层模块初始化。其次是 WSL 环境下 RViz 渲染问题, 在 WSL2 中运行 RViz2 时可能出现 OpenGL 渲染异常, 解决方案是设置环境变量 LIBGL_ALWAYS_SOFTWARE=1 启用软件渲染模式。再次是 Gazebo 残留进程问题, 多次运行实验后可能残留 Gazebo 进程导致端口占用, 解决方案是在启动脚本中加入 pkill -9 -f "gz sim" 清理残留进程。最后是 SLAM 与 Nav2 的定位冲突问题, 在建图阶段 SLAM 提供定位功能, 此时不应启动 AMCL, 解决方案是在 slam_demo.launch.py 中使用 navigation_launch.py, 而在 nav_demo.launch.py 中使用 bringup_launch.py。

第八章 实验总结

本实验基于 ROS2 Jazzy 和 Gazebo 仿真环境，以 TurtleBot3 Burger 为平台，成功实现了移动机器人的自主建图与多点导航系统。在自主建图方面，通过集成 `slam_toolbox` 和 `explore_lite`，实现了未知环境下的自主探索建图，前沿探索算法能够高效地驱动机器人完成全图探索，探索完成后自动返回起点。在多点导航方面，通过 Nav2 导航栈和自定义控制节点，实现了已知环境下的多点序列导航，AMCL 定位稳定，路径规划合理，四个目标点均成功到达。在系统集成方面，通过 Launch 文件和 Shell 脚本实现了系统各模块的自动化启动与流程控制，简化了操作流程，提高了实验效率。在工程实践方面，深入理解了 ROS2 的节点通信机制、TF 坐标变换、参数配置等核心概念，积累了机器人系统开发与调试的实践经验。

本实验的不足之处在于仿真环境相对简单，未在真实机器人上验证；前沿探索算法的参数需要针对不同环境进行调整；多点导航的目标点为硬编码，缺乏动态规划能力。未来可考虑引入更高级的探索策略和动态目标点规划算法，进一步提升系统的智能化水平。